INT161 · Basic Backend Development

Mock Exam

ซ้อมสอบปฏิบัติ: endpoint ที่ต้องทำ วิธีให้คะแนน และการทดสอบเองแบบ offline

Exam Format | Setup | Endpoints and Scoring | Self-Testing | Submission | อภิธานศัพท์

01

Exam Format

อาจารย์เรียกว่าซ้อมหนีไฟ ตอนซ้อมสบาย แต่ของจริงรู้สึกต่างกัน เป้าหมายคือให้รู้ขั้นตอน 1, 2, 3 ก่อนที่จะนับคะแนนจริง

- คุณจะได้ใบงานเฉพาะบุคคล เป็นไฟล์ markdown และ **project template** ใบงานมีรหัสนักศึกษาและ secret key ของคุณ เพื่อให้ตัวตรวจรู้ว่าเป็นงานของใคร
- สอบปฏิบัติคือโปรแกรมที่ต้องทำงานได้ เปิดใบงานโหมด preview และอ่านให้ครบก่อน **endpoint** ที่ไม่ตรงตามใบงานจะไม่ได้คะแนน
- ข้อสอบเป็นแบบ offline รัน **npm** install ได้ครั้งเดียวตอนที่ยังออนไลน์ หลังจากนั้นให้ปิดเน็ต ถ้าเปิดกลับมาถือว่าทุจริต
- ตัวตรวจรันบนเครื่องคุณเองและพิมพ์คะแนน คุณจึงรู้ผลก่อนส่ง

> **กรณีตัวอย่าง**
> ระบบตัวอย่างคือ Mobile Phone Distributor: brands กับ products แบบ one to many วันสอบหัวข้ออาจต่างไป แต่อาจารย์คาดว่ายังเป็นสองตารางและมี one to many หนึ่งเส้น

02

Setup

ขั้นแรกคือได้ database คุณจะได้ไฟล์ SQL ที่สร้างและใส่ข้อมูลให้ ถ้ารันสคริปต์นี้ไม่ได้ ก็สอบไม่ได้

import the database

```
$ mysql -u root -p < database/init-database.sql
```

- ถ้าไม่เจอคำสั่ง mysql แปลว่า MySQL ไม่อยู่ใน **PATH** เพิ่มโฟลเดอร์ bin ใน PATH หรือข้ามการใช้ command line
- วิธีที่เชื่อถือได้ใน IDE: สร้าง data source คลิกขวา Run SQL Script เลือกไฟล์ แล้ว refresh จะเห็น database mobile_distributor พร้อมสองตาราง
- สร้าง data source ไว้ก่อนวันสอบ เพราะวันนั้นอาจยังไม่มี **connection**
- ตั้งค่า connection ใน src/configs/db-config.js ให้ตรงกับเครื่อง โดยใช้ database mobile_distributor
- แตก zip ของโปรเจกต์และเช็คว่าโฟลเดอร์ไม่ซ้อนกันสองชั้น

ขั้นที่สอง: ใส่รหัสนักศึกษาใน app.js และวางทุก **route** ภายใต้ /api/รหัสของคุณ

src/app.js

```
const STUDENT_ID = 'YOUR_STUDENT_ID';
// every route lives under /api/<your id>/...
app.get(`/api/${STUDENT_ID}/student-info`, (req, res) => {
  res.status(200).json({
    success: true,
    data: { studentId: STUDENT_ID, name: 'YOUR_NAME', secretKey: 'YOUR_SECRET_KEY' },
    timestamp: new Date().toISOString(),
  });
});
```

- ถ้า base path ผิด ตัวตรวจจะเข้า route ของคุณไม่ได้และทุกข้อไม่ผ่าน
- ตัวตรวจเทียบรหัสและ secret key กับตารางของมัน ค่าที่ผิดนับว่าผิด

> ⚠ **ค่าแทน**
> code ตรงนี้ใช้ YOUR_STUDENT_ID และ YOUR_SECRET_KEY รหัสและ key จริงอยู่ในใบงานของคุณ เป็นข้อมูลเฉพาะบุคคล ห้ามใช้ของคนอื่น

03

Endpoints and Scoring

ทุกอย่างที่ต้องใช้สอนมาแล้ว นี่คือที่มาของแต่ละส่วนในใบงาน

`Identity endpoint`

Class 5: a route that returns JSON.

`Brands GET`

Classes 4 and 6: repository, service, route; 404 when missing.

`Products list`

Class 8: page, limit, offset, summary DTO and metadata.

`Product detail`

Class 8: detail DTO. A join brings in the brand name.

`POST with duplicate SKU`

Class 7: duplicate entry becomes 409.

`PUT and DELETE`

Classes 4 and 6: 200 or 204, and 404 when missing.

- ยืนยันตัวตน: GET /api/id/student-info ตอบ 200 พร้อม success, data (studentId, name, secretKey) และ **timestamp**
- Brands: GET /brands ได้รายการ GET /brands/:id ได้หนึ่งรายการ หรือ 404 ถ้าไม่มี แต่ละ brand มี id, name, country_of_origin, website, is_active และ created_at ซึ่งตรงกับ table อยู่แล้ว
- รายการสินค้า: GET /products พร้อม page (ค่าเริ่มต้น 1) และ **limit** ใช้ **summary DTO** เท่านั้น: id, sku, name, brandId, category, price, status

Products list response

```
{
  "success": true,
  "data": [
    { "id": 1, "sku": "APL-IP15PM-256-NT", "name": "iPhone 15 Pro Max",
      "brandId": 1, "category": "FLAGSHIP", "price": 48900.00, "status": "AVAILABLE" }
  ],
  "pagination": {
    "total": 75, "page": 1, "limit": 10, "totalPages": 8,
    "hasNextPage": true, "hasPrevPage": false
  },
  "timestamp": "2026-10-08T15:00:00.000Z"
}
```

- รายละเอียดสินค้า: field ของ summary บวก brandName, ramGb, storageGb, color, stockQuantity และ releaseDate brandName อยู่ในอีกตาราง จึงต้อง join ถ้าไม่มี ตอบ 404
- POST /products: 201 เมื่อบันทึกสำเร็จ 409 เมื่อ SKU ซ้ำ **request body** ใช้ชื่อแบบ **snake_case** การทำ DTO จึงไปทั้งสองทาง
- PUT /products/:id: 200 เมื่ออัปเดตได้ 404 เมื่อไม่มี id นั้น
- DELETE /products/:id: 204 และไม่มี body หรือ 404 เมื่อไม่มี id นั้น

POST /products request body

```
{
  "sku": "TEST-SKU-999", "name": "Test Phone 2026", "brand_id": 1,
  "category": "FLAGSHIP", "price": 29900.00, "ram_gb": 8, "storage_gb": 256,
  "color": "Black", "stock_quantity": 10, "status": "AVAILABLE",
  "release_date": "2026-01-01"
}
```

ติ๊กข้อที่ผ่านเพื่อรวมคะแนนที่ระบุรายข้อ

- [ ] GET /student-info **10**
- [ ] GET /brands and /brands/:id **10**

☐	`GET /products (page, limit, summary DTO)`	**30**
☐	`GET /products/:id (detail DTO)`	**10**
☐	`POST /products`	**10**
☐	`PUT /products/:id`	**10**
☐	`DELETE /products/:id`	**5**

คะแนนที่ระบุรายข้อ: 0 จาก 85

แต่ละ **endpoint** ให้คะแนนแยกกัน โปรเจกต์ที่ทำได้ครึ่งเดียวก็ยังได้คะแนน ใบงานบอกว่าคะแนนเต็มคือ 100 แต่รวมแต่ละส่วนได้ 85 อีก 15 ไม่ได้แจกแจง

04

Self-Testing

รัน **server** ที่ **port** 3000 แล้วรันตัวตรวจในอีก terminal โดยใส่รหัสของคุณเอง

two terminals

```
# terminal 1
$ npm start
# terminal 2
$ node tests/test-runner.mjs YOUR_STUDENT_ID
```

- ตัวตรวจใช้ไฟล์ทดสอบเดียวกันทุกคน แต่เรียก endpoint ของรหัสที่คุณใส่
- พิมพ์ผลของทุกข้อและคะแนนรวม แต่ละข้อให้คะแนนแยกกัน โปรเจกต์ที่ทำได้เพียงบางส่วนก็ยังได้คะแนน
- ข้อที่ไม่ผ่านจะพิมพ์ Reason เช่น expected 200 แต่ได้ 404 อ่านแล้วเช็ค **status code** และ field ใน body แก้ทีละอย่างแล้วรันใหม่
- เวลาไม่พอ เริ่มจากข้อ 1 เป็น route เล็ก ๆ ที่ได้ 10 คะแนน

ลำดับการทำงาน

ทำทีละ layer และทดสอบไปเรื่อย ๆ เหมือนคลาส 6 ตัวตรวจจะแค่ยืนยันสิ่งที่คุณรู้อยู่แล้ว

05

Submission

เช็คสิ่งเหล่านี้ก่อนส่ง

1. app.js มีรหัสนักศึกษาของคุณเอง
2. endpoint ยืนยันตัวตนตอบชื่อและ secret key ของคุณ
3. ตัวตรวจผ่านทุกข้อเท่าที่ทำได้บนเครื่องคุณ
4. บีบอัดเฉพาะโฟลเดอร์งาน ลบ node_modules ออก และตั้งชื่อ รหัสของคุณ_exam.zip

> **ซ้อมจริง**
> ในคลาสไม่ได้ส่งจริง ถ้าไม่มั่นใจ มีเวลาสองสัปดาห์ให้ซ้อมทั้งกระบวนการ ทำตามลำดับ: นำเข้า database, ตั้งรหัส, สร้าง, รันตัวตรวจ, zip

06

อภิธานศัพท์

body
: ข้อมูลที่ request พกมาหลัง header สำหรับ API มักเป็น JSON

connection
: ช่องทางที่เปิดอยู่ระหว่างโปรแกรมกับ database ต้องมีก่อนส่ง SQL ได้

DTO
: สำเนาของ entity ที่ปรับรูปสำหรับ API มีเฉพาะ field ที่ client ต้องการ ในรูปแบบที่ตกลงกัน

endpoint
: method กับ path หนึ่งคู่ที่ API ตอบ เช่น GET /api/subjects

limit
: จำนวนรายการต่อหนึ่งหน้า client ส่งมาเป็น limit และ server กำหนดเพดาน

npm
: Node Package Manager: ติดตั้ง library เช่น express ลงใน node_modules

PATH
: ค่าตั้งของ Windows ที่ระบุโฟลเดอร์ซึ่งใช้ค้นหาคำสั่งที่พิมพ์ ถ้า bin ของ MySQL ไม่อยู่ในนี้ จะไม่รู้จักคำสั่ง mysql

port

เลขที่บอกว่าโปรแกรมไหนบนเครื่องควรได้รับข้อความ web server ใช้ 80 เป็นค่าเริ่มต้น

project template

โปรเจกต์ตั้งต้นที่บันทึกไว้ โปรเจกต์ใหม่ที่สร้างจากมันจะมีโฟลเดอร์และ code พื้นฐานให้แล้ว

request

สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

route

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

server

โปรแกรม (หรือเครื่องที่รันมัน) ที่รอรับ request แล้วตอบกลับ

snake_case

คำต่อกันด้วย underscore เช่น user_name ใช้บ่อยกับคอลัมน์ใน database ไม่ใช้กับ URI

status code

ตัวเลขที่บอก client ว่าผลเป็นอย่างไร เช่น 200 OK, 201 Created, 404 Not Found, 500 Server Error

summary

DTO สองขนาด: summary มีไม่กี่ field สำหรับรายการ detail มีครบสำหรับรายการเดียว

timestamp

ค่าวันที่และเวลา คอลัมน์อย่าง last_update ใส่เวลาปัจจุบันให้เองได้